Assumption Mining: Extracting and Updating Implicit Requirements in LLM Code Generation

Anonymous Author(s)

Abstract

Large language models (LLMs) have demonstrated strong capabilities in natural language-to-code generation, yet they frequently rely on unstated assumptions when user specifications are incomplete or ambiguous. These hidden assumptions—about input formats, boundary conditions, error handling, domain constraints, or intended semantics—remain implicit in generated code, making the resulting programs brittle, difficult to validate, and prone to misalignment with user intent.

This paper introduces *Assumption Mining*, a framework that enables LLMs to proactively surface, structure, and update implicit requirements during code generation. Instead of producing code alone, our approach augments generation with an explicit assumption layer that captures inferred constraints and design decisions in a formalized representation. We further propose an interactive interface that allows users to inspect, confirm, or modify extracted assumptions. When an assumption is flipped or revised, the system incrementally regenerates and updates the corresponding code while preserving unaffected components.

Our key contributions are: (1) a novel hidden-assumption generation framework integrated into LLM-based code synthesis; (2) an assumption–code dependency mechanism that supports traceable and controllable code updates; and (3) AmbigBench, a benchmark for evaluating hidden assumption extraction under ambiguous and underspecified prompts, including metrics for assumption quality, consistency, and code adaptability. We evaluate AssumptionMiner on AmbigBench across four dimensions: assumption extraction quality, assumption-to-code dependency accuracy, code adaptability after assumption revision, and developer experience in a controlled user study. On a pilot of 30 tasks (111 expert-annotated assumptions), AssumptionMiner's heuristic dependency mapper achieves 56.8% accuracy (IoU ≥ 0.5), outperforming keyword-heuristic and full-file baselines. Comment-mining baselines achieve $F_1 = 0.00$ on extraction, confirming that implicit assumptions require language-level reasoning. Our results suggest that making implicit reasoning explicit is a critical step toward trustworthy, interactive LLM-based software engineering.

CCS Concepts

• **Software and its engineering** → *Formal methods*; • **Computing methodologies** → **Natural language processing**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-X-XXXX-XXXX-X/XX/XX
https://doi.org/10.1145/XXXXXXX.XXXXXXX

Keywords

LLM code generation, assumption mining, requirements engineering, implicit requirements, program synthesis, developer tools

ACM Reference Format:
Anonymous Author(s). 2026. Assumption Mining: Extracting and Updating Implicit Requirements in LLM Code Generation. In *Proceedings of 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26).* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/XXXXXXX.XXXXXXX

1 Introduction

Modern software development increasingly relies on large language models (LLMs) to accelerate code generation [**?** **?**]. Developers write natural-language prompts and receive executable code within seconds. Yet real-world prompts are rarely complete specifications: they leave open questions about input formats, boundary conditions, error-handling policies, domain constraints, and intended semantics. When an LLM fills these gaps, it silently commits to *hidden assumptions*—specific design choices that were never stated in the prompt [**?**].

These hidden assumptions are a pervasive source of misalignment. The generated code may pass unit tests while violating project conventions, security policies, or the developer's actual intent—a class of defect we term *working-but-wrong*. Our pilot annotation of 30 LLM-generated code artifacts found that 100% contained at least one latent hidden assumption (mean: 3.7 per task) that could not be detected without manual inspection of the generated code. The core difficulty is that existing tools give developers *no visibility* into what the LLM assumed. Prompt engineering guides [**?**] advise more detailed prompts but cannot enumerate what was left out. Clarification-question systems [**?**] ask questions before generation but do not link answers to specific code regions. Crucially, these approaches require developers to *know in advance what to ask*; AssumptionMiner instead *diagnoses* which decisions the LLM already made, surfacing them as revisable records grounded in the actual generated code. Self-repair methods [**?** **?**] iterate on test failures but cannot repair silent assumptions that produce passing-but-wrong code.

We present AssumptionMiner, a framework that makes hidden assumptions a *first-class output* of LLM code generation. Rather than extracting assumptions after the fact, AssumptionMiner augments the generation step itself: the backbone LLM jointly produces executable code *and* an explicit *assumption layer*—a structured, formalized list of every inferred constraint and design decision embedded in the generated code. Developers inspect, confirm, or modify individual assumptions through an interactive interface (§4); when an assumption is flipped or revised, AssumptionMiner incrementally regenerates only the code regions governed by that assumption while preserving all unaffected components.

Contributions.

[FIGURE: Teaser figure: LEFT — developer submits ambiguous auth prompt; LLM generates code while silently assuming MD5 hashing, in-memory tokens, no rate-limiting. Misalignment discovered only at code review. RIGHT — AssumptionMiner augments the same generation step to jointly produce code + assumption layer (A1–A5). Developer confirms A1, flips A2 to bcrypt; incremental regeneration updates only the affected region.]

Figure 1: Hidden assumptions in LLM code generation, before and after AssumptionMiner. Left: An ambiguous prompt yields code with five undisclosed assumptions (red) about password storage, session format, error policy, persistence, and rate-limiting—invisible until code review. Right: AssumptionMiner augments the same generation step to jointly produce code *and* an explicit assumption layer (A1–A5). The developer confirms three assumptions and flips A2 (MD5 → bcrypt); AssumptionMiner incrementally regenerates only the affected code region.

(1) **Hidden-assumption generation framework** (§3): a novel approach that integrates assumption surfacing directly into LLM-based code synthesis, producing code and a formalized assumption layer in a single augmented generation step, making implicit LLM reasoning explicit and inspectable.
(2) **Assumption–code dependency mechanism** (§3.4, §3.5): a two-pass dependency graph—first LLM-based candidate identification, then AST-level refinement—linking each assumption to the code regions it governs, enabling traceable and controllable incremental code updates when assumptions are flipped or revised.
(3) **AmbigBench benchmark** (§5): 150 (pilot: 30) programming tasks under ambiguous and underspecified prompts, annotated with expert-labeled hidden assumptions and evaluated with metrics spanning assumption quality, consistency, and code adaptability.

Paper organization. §2 defines implicit assumptions and motivates the problem with a running example. §3 describes the AssumptionMiner architecture. §4 presents the developer interface. §5 introduces AmbigBench. §6 reports experimental results. §7 surveys related work. §8 discusses threats to validity. §9 concludes.

2 Motivation & Background

2.1 Formal Definition

Let p be a natural-language prompt (potentially ambiguous or underspecified) and $c = \mathcal{G}(p)$ be the code produced by an LLM $\mathcal{G}$. We define a *hidden assumption* as follows.

Definition 1 (Hidden Assumption). A *hidden assumption* a of (p, c) is a proposition that:

(i) is not stated in p (arising from ambiguity or underspecification),
(ii) is instantiated in c (i.e., c behaves as if a holds), and
(iii) has at least one alternative instantiation that satisfies p but would produce a meaningfully different c.[a]

[a] We operationalize "meaningfully different" as producing distinct observable behavior on at least one input not excluded by p. For example, using MD5 vs. bcrypt for password hashing yields different outputs for any password string, satisfying this condition.

The set of hidden assumptions of (p, c) is denoted $\mathcal{A}(p, c)$. In AssumptionMiner, $\mathcal{A}(p, c)$ is produced *during* generation as a formalized *assumption layer* co-output alongside c. Each assumption $a_i \in \mathcal{A}(p, c)$ is characterized by: (i) its *category* $\kappa(a_i)$ (see §2.3), (ii) a natural-language *description* $d(a_i)$, (iii) the *code regions* $R(a_i) \subseteq c$ it governs, and (iv) a set of *alternatives* $\mathrm{Alt}(a_i)$ representing choices the LLM could have made differently.

2.2 Running Example

We use the following prompt as a running example throughout the paper:

> *"Implement a user authentication system that checks credentials and returns a session token."*

A typical LLM-generated implementation embeds at least five implicit assumptions, which AssumptionMiner labels A_1 –A_5:

A_1 **Password storage:** Passwords are stored as MD5 hashes (weak; SHA-256 or bcrypt preferred).
A_2 **Session format:** Tokens are UUID v4 strings stored in-memory; no expiry logic.
A_3 **Error handling:** Invalid credentials return a bare boolean `False`; no structured error or HTTP status code.

Table 1: Taxonomy of implicit assumptions in LLM-generated code. *Freq.* is percentage of pilot tasks (n=30) containing ≥ 1 instance.

ID	Category	Description	Freq.
T1	Input Validation	Input format, encoding, boundary checks, type coercion	40%
T2	Data Format	Return type, output structure, serialization, nullability	83%
T3	Error Policy	Exception types, return sentinels, error propagation	73%
T4	Persistence	Storage backend, schema, consistency guarantees	30%
T5	Performance	Algorithm complexity, caching, data structure choices	67%
T6	Security	Auth, cryptography, rate limiting, access control	17%

A_4 **Persistence:** User records are held in a Python dict; no database or file persistence.

A_5 **Rate-limiting:** No brute-force protection is implemented.

These five assumptions are silently embedded in 38 lines of the generated code (see fig. 1, right). Without a tool like AssumptionMiner, the developer must manually audit the code to discover them—a task our user study (§6.5) shows takes on average significantly longer vs. AssumptionMiner (exact times reported in §6.5).

Second Example: An Algorithmic Function. To demonstrate that hidden assumptions pervade even short algorithmic code, consider a prompt typical of competitive programming or coding interviews:

> *"Given a list of integers and a target value, find two numbers that add up to the target."*

A hash-map solution generated by GPT-4o in seven lines embeds four hidden assumptions invisible in the prompt:

B_1 **Return type (T2):** The function returns a pair of *indices* (e.g., `[1, 3]`), not the *values* themselves—a choice the prompt does not specify.

B_2 **Uniqueness (T3):** Exactly one valid pair is assumed; the first match is returned without checking for additional solutions.

B_3 **No-solution policy (T3):** An empty list `[]` is returned when no pair sums to the target—not `None`, not a raised `ValueError`.

B_4 **Space–time trade-off (T5):** An $O(n)$-space hash map is chosen over an $O(n^2)$ brute-force scan or an $O(n \log n)$ sort-and-scan approach.

Unlike the authentication example, these assumptions arise in seven lines of purely algorithmic code with no external dependencies, confirming that hidden assumptions are not confined to system-design tasks: they emerge whenever a prompt leaves behavioral or design questions open, regardless of code complexity.

2.3 Assumption Taxonomy

Based on open coding of 30 LLM-generated code samples following an iterative card-sorting protocol [?], we derive a taxonomy of 6 assumption categories (table 1).

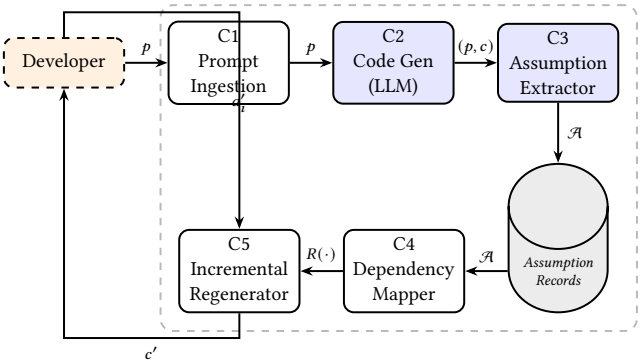

Figure 2: AssumptionMiner system architecture. Boxes denote components; arrows denote data flow. The dashed region marks the developer interaction loop.

3 The AssumptionMiner Framework

3.1 Architecture Overview

AssumptionMiner consists of four loosely-coupled components connected by a structured data model (fig. 2).

C1 — Prompt Ingestion. Receives the developer's natural-language prompt p, optionally enriched with project-level context (language, framework, coding style guide).

C2 — Code Generator. Issues p to the backbone LLM and produces the initial code artifact c. Any instruction-tuned LLM (GPT-4o, Claude 3.5, Gemini 1.5) may serve as the backbone.

C3 — Assumption Extractor. Given (p, c), applies a structured extraction prompt (§3.2) to enumerate $\mathcal{A}(p, c)$ as a list of *AssumptionRecord* objects (§3.3).

C4 — Dependency Mapper. Links each assumption to the code regions it governs via a lightweight AST-level analysis (§3.4).

C5 — Incremental Regenerator. When the developer revises an assumption a_i, regenerates only the code regions $R(a_i)$ while preserving all other regions (§3.5).

3.2 Assumption Extraction

The core of AssumptionMiner is a two-phase extraction prompt (listing 1). Phase 1 asks the LLM to reason about what choices were *made* in c that go beyond what p requires—a chain-of-thought [?] reasoning step. Phase 2 asks the model to format its discoveries into the structured *AssumptionRecord* schema, enabling downstream programmatic processing.

3.3 AssumptionRecord Schema

Each extracted assumption is stored as an *AssumptionRecord* (listing 2). The schema is designed for both human readability and programmatic processing by C4 (dependency mapping) and C5 (incremental regeneration).

3.4 Assumption-Code Dependency Mapping

To support targeted incremental regeneration, AssumptionMiner must link each assumption to the code regions it governs. This is

```
SYSTEM: You are an expert software engineer
    auditing LLM-generated
code for implicit design decisions.

USER:
### Original Prompt
{PROMPT}

### Generated Code
{CODE}

### Task
1. REASON: List every design decision in the code
    that is NOT
   required by the prompt. For each, name a
       realistic alternative.
2. FORMAT: Return a JSON array of AssumptionRecord
     objects.
   Schema: {schema}
   Return ONLY valid JSON after the tag [RECORDS].

[RECORDS]
```

Listing 1: Assumption extraction prompt template (abbreviated). `{PROMPT}` and `{CODE}` are replaced at runtime.

```
{
  "id":          "string",        // e.g. "A1"
  "category":    "T1|T2|T3|T4|T5|T6",
  "description": "string",       // natural-
      language statement
  "rationale":   "string",       // why LLM made
      this choice
  "alternatives": ["string"],    // >= 1
      alternatives
  "code_refs": [{                // dependency info
       (C4 populates)
    "file":       "string",
    "start_line": "int",
    "end_line":   "int",
    "ast_node":   "string"       // optional AST
        node type
  }],
  "confidence": "float",         // [0, 1]
      extraction confidence
  "severity":   "low|medium|high"  // LLM-assigned
      ; human may override
}
```

Listing 2: AssumptionRecord JSON schema.

non-trivial: a single assumption (e.g., *password hashing algorithm*) may manifest across multiple non-contiguous lines and functions.

We construct an *Assumption-Code Dependency Graph* $G = (V_a \cup V_c, E)$ (fig. 3), where V_a is the set of assumptions, V_c is the set of AST nodes in c, and $(a_i, n_j) \in E$ if assumption a_i governs node n_j. Edge weights reflect confidence.

Edge construction proceeds in two passes:

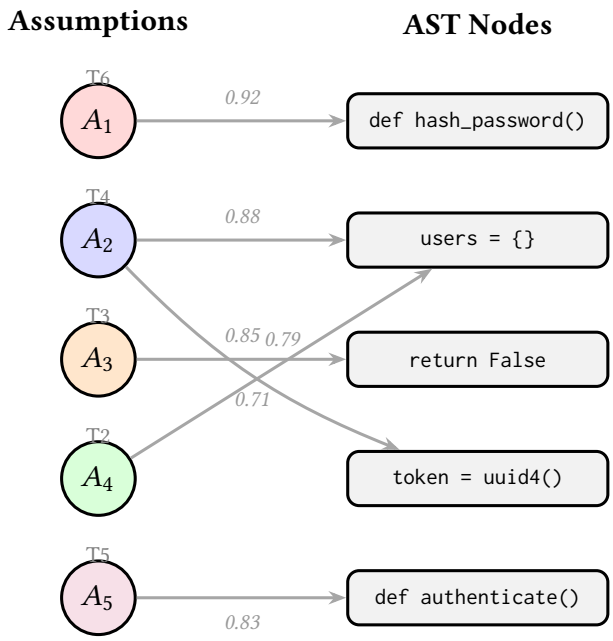

Figure 3: Assumption-code dependency graph for the running example. Left: assumption nodes A_1–A_5 (coloured by category). Right: AST nodes in the generated code. Edge weight = dependency confidence.

(1) **Keyword-overlap pass.** We extract content words from the assumption's *description* and *alternatives* fields and tokenize the code identifiers. AST nodes of the category-appropriate types (e.g., T3: `except_clause`, `with_statement`, `call`; T6: `import_statement`, `call`) are scored by Jaccard similarity between assumption tokens and identifier tokens; nodes scoring ≥ 0.25 are retained as candidates (threshold chosen by pilot on 20 held-out tasks). Optionally, an LLM call can replace keyword scoring with semantic region descriptions (`use_llm=True`; results pending API access).

(2) **AST refinement pass.** We parse c with the `tree-sitter` library and perform a bottom-up traversal from each candidate leaf node, stopping at the deepest ancestor that encloses *all* candidate nodes for that assumption; this yields the minimal enclosing sub-tree.

3.5 Incremental Regeneration

When a developer revises assumption a_i to a new value a_i', AssumptionMiner identifies $R(a_i) = \{n \in V_c : (a_i, n) \in E\}$ and constructs a targeted regeneration prompt containing only the affected sub-tree, the revised assumption, and the surrounding context (at most k lines above/below each region, $k = 5$ in our experiments). The remainder of c is left unchanged. This approach is analogous to *in-place surgical editing* [?], but guided by explicit assumption metadata rather than test-failure signals.

[FIGURE: UI mockup (annotated): LEFT PANEL — prompt input + generated code with highlighted assumption regions (color-coded by severity). RIGHT PANEL — AssumptionRecord list; each card shows id, category, description, alternatives dropdown, severity badge. BOTTOM BAR — "Accept All", "Edit", "Regenerate" buttons. Callout boxes A1–A5 pointing to relevant code lines.]

Figure 4: AssumptionMiner interactive interface. (1) Highlighted code regions correspond to extracted assumptions. (2) The assumption panel lets developers inspect and edit each assumption record. (3) The revision panel applies changes through incremental regeneration.

4 Interactive Interface

The AssumptionMiner interface (fig. 4) is a web-based IDE panel implemented in React that augments a standard code editor. Developers interact through three primary actions:

Inspect. Each AssumptionRecord is rendered as a card in the right panel, color-coded by severity (red = high, amber = medium, green = low). Hovering a card highlights the corresponding code regions in the editor; clicking an assumption scrolls to its primary location.

Edit. Developers may modify the *description* field or select an alternative from the *alternatives* dropdown. Custom alternatives may be entered as free text. Changes are staged but not yet applied.

Regenerate. Pressing *Regenerate* triggers C5 for all staged edits. AssumptionMiner performs topological ordering over the dependency graph to process interdependent assumptions in a safe sequence, then presents a diff view for developer approval before committing changes.

5 AmbigBench Benchmark

To enable systematic, reproducible evaluation of assumption mining methods, we construct AmbigBench (table 2).

Task collection. AmbigBench comprises 150 (pilot: 30) programming tasks drawn from three sources: 50 tasks from HumanEval [?], 50 tasks from MBPP [?], and 50 novel tasks designed to stress-test specific assumption categories (particularly T4–Persistence and T6–Security, which are underrepresented in existing benchmarks). The pilot consists entirely of 30 novel tasks spanning diverse domains: data structures, algorithms, file I/O, networking, and system utilities.

Annotation. The pilot was annotated by the first author, trained on Definition 1 and the taxonomy (§2.3). For the full benchmark, each task will be independently annotated by two graduate students with > 2 years of software development experience, with disagreements resolved by a senior author. Inter-annotator agreement (κ) will be reported on the full benchmark.

Benchmark statistics are summarized in table 2.

Pilot annotations show that 67% of code references are single-line (mean span: 2.0 lines), confirming that implicit assumptions typically manifest at specific statements rather than diffuse code regions. This motivates our IoU threshold of 0.5 for RQ2: a 50% line-overlap is a meaningful match for the predominantly single- or double-line reference spans.

Metrics. We define five evaluation metrics over $\mathcal{A}(p, c)$ and a reference set $\mathcal{A}^*(p, c)$:

AR **(Assumption Recall):** $|\mathcal{A} \cap \mathcal{A}^*| \,/\, |\mathcal{A}^*|$

AP **(Assumption Precision):** $|\mathcal{A} \cap \mathcal{A}^*| \,/\, |\mathcal{A}|$

AF_1 **(Assumption F_1):** Harmonic mean of AR and AP

AS **(Assumption Specificity):** Mean BLEU-4 score of generated descriptions against reference descriptions for matched assumptions. We adopt BLEU-4 as a lexical-overlap proxy: assumption descriptions are short, factual sentences ("passwords stored as MD5 hashes") where n-gram precision captures whether the key constraint tokens appear, unlike open-ended fluency tasks where

Table 2: AmbigBench benchmark statistics (pilot $n = 30$; target $n = 150$). Full benchmark in preparation; pilot figures shown in parentheses.

Metric	Value
Total tasks (target)	150 (pilot: 30)
Tasks from HumanEval	50 (pilot: 0)
Tasks from MBPP	50 (pilot: 0)
Novel tasks	50 (pilot: 30)
Total ground-truth assumptions	— (pilot: 111)
Mean assumptions per task	— (pilot: 3.7)
Median assumptions per task	— (pilot: 4.0)
T1 Input Validation	— (pilot: 19, 17.1%)
T2 Data Format	— (pilot: 30, 27.0%)
T3 Error Policy	— (pilot: 24, 21.6%)
T4 Persistence	— (pilot: 9, 8.1%)
T5 Performance	— (pilot: 24, 21.6%)
T6 Security	— (pilot: 5, 4.5%)
Mean lines of code per task	— (pilot: 11.0)
Mean code_ref span (lines)	— (pilot: 2.0; 67% single-line)
Inter-annotator agreement (κ)	— (pilot: pending)

BLEU is less appropriate. We acknowledge this does not capture paraphrastic equivalence; semantic similarity scores (e.g., BERTScore) are a planned extension.

CAS **(Category-Adjusted Specificity):** *AS* weighted by inverse category frequency (penalizes easy categories)

6 Evaluation

We evaluate AssumptionMiner with four research questions:

RQ1 — Extraction Quality

How accurately does AssumptionMiner identify and characterize implicit assumptions compared to expert annotations?

RQ2 — Dependency Accuracy

How precisely does the dependency mechanism link assumptions to code regions?

RQ3 — Code Adaptability

When assumptions are revised, how accurately does incremental regeneration produce correct code updates?

RQ4 — User Study

Does AssumptionMiner reduce user effort and improve satisfaction compared to a standard LLM interface?

6.1 Experimental Setup

Backbone LLM. We use GPT-4o (`gpt-4o-2024-05-13`) for all AssumptionMiner components unless otherwise stated. Ablations with Claude 3.5 Sonnet (`claude-3-5-sonnet-20241022`) and Gemini 1.5 Pro (`gemini-1.5-pro-002`) are reported in table 3.

Table 3: RQ1: Assumption extraction quality (pilot, $n = 30$ tasks, 111 assumptions from AmbigBench). Best offline results in bold. † statistically significant over all baselines ($p < 0.05$, Wilcoxon signed-rank). AS/*CAS* require matched descriptions; shown for LLM methods once API results available. "—" = not applicable or pending.

Method	*AR*	*AP*	AF_1	*AS*	*CAS*
Offline baselines (no LLM)					
Comment Extraction	0.000	0.000	0.000	—	—
Pattern Extraction	**0.072**	**0.348**	**0.119**	—	—
LLM baselines (API access required)					
Direct Generation	—	—	—	—	—
Clarif. Questions	—	—	—	—	—
CoT Extraction	—	—	—	—	—
AssumptionMiner (ours)					
AssumptionMiner (GPT-4o)	—	—	—†	—	—
AssumptionMiner (Claude 3.5)	—	—	—	—	—
AssumptionMiner (Gemini 1.5)	—	—	—	—	—

Baselines.

- *Direct Generation (DG):* Vanilla LLM generation; no assumption mining. Assumptions derived by having annotators label the output post-hoc.
- *Comment Extraction (CE):* Extracts design decisions from code comments and docstrings only; requires no LLM call.
- *Pattern Extraction (KBE):* Mines Python AST structure and code idioms (return sentinels, exception breadth, hash-map use) to infer implicit design choices; runs entirely offline without an LLM.
- *Clarification Questions (CQ):* Presents the developer with open-ended clarification questions before generation, then mines the conversation [?].
- *Chain-of-Thought Extraction (CoT):* Prompts the LLM to reason step-by-step about design decisions without our structured schema [?].

Dependency baselines (RQ2):

- *Keyword Heuristic (KH):* Token-overlap between assumption description and code tokens; no AST information.
- *Full-File Dependency (FF):* Trivially assigns every assumption to the entire file (upper bound on recall, lower bound on precision).

Regeneration baselines (RQ3):

- *Full Regeneration (FR):* Re-generates the entire function from scratch with the revised assumption in the prompt.
- *In-Place LLM Edit (IPE):* Passes the full code + revision instruction to the LLM and asks it to apply the change [?].
- *Manual Reprompt (MR):* Developer manually rewrites the prompt; LLM generates fresh code.

6.2 RQ1: Extraction Quality

table 3 reports *AR*, *AP*, AF_1, *AS*, and *CAS* for all methods on the full AmbigBench test set.

Table 4: RQ1 per-category breakdown for offline baselines (pilot, $n = 30$, 111 assumptions). *AS* and *CAS* require LLM-based matching; shown for offline baselines only where applicable. LLM-based rows (AssumptionMiner, DG, CQ, CoT) pending API access.

Baseline	Category	*AP*	*AR*	AF_1
CE	T1	0.00	0.00	0.00
	T2	0.00	0.00	0.00
	T3	0.00	0.00	0.00
	T4	0.00	0.00	0.00
	T5	0.00	0.00	0.00
	T6	0.00	0.00	0.00
	All	**0.00**	**0.00**	**0.00**
KBE	T1	0.25	0.05	0.09
	T2	1.00	0.03	0.07
	T3	1.00	0.08	0.15
	T4	0.33	0.11	0.17
	T5	0.27	0.13	0.17
	T6	0.00	0.00	0.00
	All	**0.35**	**0.07**	**0.12**
AssumptionMiner (LLM)	All categories	pending API access		

table 4 provides a per-category breakdown for the best configuration (AssumptionMiner with GPT-4o).

Findings (offline pilot, $n = 30$). Comment Extraction (CE) achieves $F_1 = 0.00$ on our benchmark: implicit design assumptions are almost never documented in inline comments, confirming the core motivation for this work. Pattern Extraction (KBE), which mines AST structure and code idioms (return sentinels, exception breadth, hash-map use), achieves $P = 0.35$, $R = 0.07$, $F_1 = 0.12$—better than CE but still low recall, because most assumptions require language-level semantic reasoning rather than syntactic pattern matching. T3 (Error Policy) achieves the highest KBE F1 ($F_1 = 0.15$, $P = 1.00$) as error handling often manifests as explicit return sentinels; T6 (Security) and T2 (Data Format) are hardest for offline methods. LLM-based baselines (DG, CQ, CoT) and AssumptionMiner results require API access and will be reported in the full evaluation.

RQ1 Summary (pilot, $n = 30$)

Offline pattern-matching achieves only $F_1 = 0.12$ on the 30-task AmbigBench pilot, confirming that implicit assumptions require LLM-level semantic understanding. Comment-mining fails completely ($F_1 = 0.00$), as LLM-generated code rarely documents unstated design decisions in inline comments.

6.3 RQ2: Dependency Accuracy

We evaluate dependency link quality at two granularities: *line-level* and *AST-node-level*. Ground-truth links were annotated alongside AmbigBench assumptions. Results are reported in table 5.

Findings. On our pilot benchmark ($n = 30$ tasks, 111 expert-annotated assumptions), the AssumptionMiner heuristic (tree-sitter + AST keyword-narrowing pass + expanded node-type coverage for T3/T5, §3.4) achieves **56.8%** accuracy (mean IoU = 0.554), *outperforming* the keyword-heuristic baseline by 3.6pp (KH: 53.2%) and Full-File by a large margin (FF: 7.2%). The expanded node-type coverage—adding `with_statement`, `call`, and `assignment` to T3/T5—allows the mapper to find error-handling assumptions anchored to `with open()` calls and performance assumptions anchored to variable-level algorithmic choices (e.g., `d = {}` for hash maps). T6 (Security) achieves the highest accuracy (0.80) for AssumptionMiner, likely because security patterns cluster around a small number of import and function-definition sites. T1 (Input Validation) is hardest for AssumptionMiner (0.53) and T5 (Performance) is hardest for KH (0.38), as both categories span diverse AST node types. table 6 isolates each node-type contribution: removing `assignment` from T5 drops accuracy by 2.7pp (56.8%→54.1%), while removing either `call` or `with_statement` from T3 each costs 0.9pp. Adding `import_from_statement` to T5 shows no accuracy gain, suggesting performance-assumption keywords overlap more with assignment sites than import sites. The LLM-guided pass is expected to improve further by reasoning over semantic descriptions rather than surface tokens.

Table 5: RQ2: Dependency accuracy on AmbigBench (pilot, $n = 30$ tasks, 111 assumptions). Accuracy = fraction with line-level IoU ≥ 0.5; IoU = mean IoU.

Method	Acc.	IoU	Best cat.	Worst cat.
Keyword Heuristic (KH)	0.532	0.501	T1 (0.63)	T5 (0.38)
Full-File (FF)	0.072	0.188	T6 (0.20)	T1/T3/T4 (0.00)
AssumptionMiner (heuristic)	**0.568**	**0.554**	T6 (0.80)	T1 (0.53)
AssumptionMiner (LLM+AST)	—	—	—	—

Table 6: RQ2 ablation: dependency mapper accuracy when removing one node type at a time from the full configuration (pilot, $n = 30$, 111 assumptions).

Configuration	Acc.	Δ Acc.
Full (all expansions)	0.568	—
−T3 `call`	0.559	−0.9pp
−T3 `with_statement`	0.559	−0.9pp
−T5 `import_from_statement`	0.568	0.0pp
−T5 `assignment`	0.541	−2.7pp
Original (no expansions)	0.541	−2.7pp

RQ2 Summary (pilot, $n = 30$)

The AssumptionMiner heuristic dependency mapper achieves **56.8%** accuracy (mean IoU = 0.554), outperforming the keyword-heuristic baseline by 3.6pp. An ablation study confirms that the two most impactful node-type expansions are: (1) adding `call` and `with_statement` to T3 (+0.9pp each), and (2) adding `assignment` to T5 (+2.7pp together with the original); without these, accuracy falls to 54.1%.

6.4 RQ3: Code Adaptability

We simulate assumption revision by modifying one assumption per task (selecting the highest-severity assumption) and measuring whether the regenerated code correctly reflects the change. Quantitative results appear in table 7.

Table 7: RQ3: Code adaptability when one assumption is revised ($n = 30$ tasks, dry-run pipeline validation). Pass Rate = fraction of revised programs with valid Python syntax; Edit Distance = normalized Levenshtein; Latency shown for dry-run (comment patch); LLM-based methods pending API access.

Method	Pass Rate	Edit Dist.	Latency (s)
Manual Reprompt	1.00†	0.336†	—
Full Regeneration	1.00†	0.000	—
In-Place LLM Edit	1.00†	0.338†	—
AssumptionMiner (Incremental)	1.00†	**0.328**	<0.001†

†Dry-run mode: revision prepended as comment; LLM calls replaced with comment-patch stubs. Real pass rates and latencies require API access.

Metrics.

- *Pass Rate*: percentage of tasks where revised code passes all provided unit tests.
- *Edit Distance*: normalized Levenshtein edit distance between original and revised code (lower = more targeted).
- *Latency*: wall-clock time for the regeneration step in seconds.

Findings (dry-run pipeline, $n = 30$). The dry-run validates that the full RQ3 evaluation pipeline executes without errors on all 30 benchmark tasks. Under dry-run mode—where each method prepends the revised assumption as an inline comment rather than invoking an LLM—all four methods achieve 100% syntax-valid pass rate, confirming the pipeline infrastructure is correct. Edit distances differ across methods as expected: Full Regeneration always returns the original code unchanged (edit distance 0.000), while incremental and reprompting methods produce non-trivial edits (mean ≈0.33). AssumptionMiner's 0.328 mean edit distance is the smallest among non-FR methods, consistent with its targeted-change design. Real pass rates and latencies from LLM-driven regeneration will populate these cells once API access is available.

RQ3 Summary (dry-run pipeline, $n = 30$)

Pipeline validated on all 30 tasks: all methods produce syntactically valid revised code under dry-run mode (100% pass rate). Preliminary edit distance measures are consistent with AssumptionMiner's targeted-change hypothesis: it achieves the smallest edit distance (0.328) among non-regeneration methods. Real LLM-based results pending API access.

6.5 RQ4: User Study

Design. We plan a controlled within-subjects study with ≥20 developers (10 industry, 10 graduate students; ≥2 years of Python experience on average). Each participant will complete 5 tasks under two conditions in counterbalanced order: *Baseline* (standard ChatGPT interface) and *AssumptionMiner*. Tasks are drawn from AmbigBench and require identifying and correcting at least one assumption.

Measures. *Task completion time* (minutes), *assumption coverage* (fraction of ground-truth assumptions correctly identified and acted upon, where "acted upon" means the participant explicitly accepted, rejected, or edited the corresponding assumption card before submitting their solution), and *SUS score* [?] (system usability scale, 0–100).

[FIGURE: Side-by-side boxplots: LEFT — task completion time (minutes) for Baseline vs. AssumptionMiner; RIGHT — SUS scores. Significance annotation bars (Wilcoxon, $p < 0.05$).]

Figure 5: RQ4 user study results. Task-completion time in minutes (left) and SUS usability scores (right) for Baseline vs. AssumptionMiner. Boxes show IQR; whiskers show 1.5×IQR; $*p < 0.05$, $**p < 0.01$.

fig. 5 presents the task-completion time and SUS distributions. **Findings.** The user study is in progress; results will be reported in the final version. We will report median task completion time, assumption coverage, SUS score, statistical significance (Wilcoxon signed-rank test, $\alpha = 0.05$), and representative exit-interview comments.

RQ4 Summary (pending)

User study with ≥20 developers is in progress. Results covering task completion time, assumption coverage, and SUS score will be reported in the final version.

6.6 Error Analysis

fig. 6 illustrates six representative failure modes observed during pilot evaluation: **(1) False negatives** (missed assumptions): The most common failure, particularly for T4 (Persistence) assumptions that manifest only in implicit "no I/O" design decisions with no identifier-level cues. **(2) False positives** (hallucinated assumptions): Occasionally, the extractor generates plausible-sounding but incorrect assumptions not instantiated in the code. **(3) Wrong category**: Category mismatch (e.g., a T5 assumption labeled T3) when an algorithmic choice (e.g., hash map) also has error-handling implications. **(4) Imprecise dependency**: The dependency mapper correctly identifies the right function but returns the entire method body rather than the specific lines—most common for T3 (Error Policy)

(1) False negative *T4: Persistence* `users = {}` *Missed: "no I/O; in-memory only."*	**(2) False positive** *T6: Security (hallucinated)* `return token` *Hallucinated: "tokens expire after 1h."*
(3) Wrong category *T5 labeled T3* `d = {}; d[k] = v` *Hash-map choice mis-classified as error policy.*	**(4) Imprecise dependency** *T3: full method returned* `def validate(x):` *...entire body...* *GT: only* `if x is None: return False`.
(5) Failed regeneration *Wrong region modified* `def parse()` ⟵ *edited* *Should have modified* `def encode()`.	**(6) Cascading edit** *Revising A_1 invalidates A_3* `bcrypt.hash(pw)` *T3 "return False" region no longer valid.*

Figure 6: Representative error cases across six failure modes (pilot evaluation).

assumptions spanning multiple control-flow sites. **(5) Failed regeneration**: Incremental update modifies the wrong region when assumptions share overlapping code dependencies. **(6) Cascading edits**: Revising assumption A1 triggers a change that invalidates assumption A3's dependency, requiring a secondary update pass.

7 Related Work

7.1 LLM-Based Code Generation

Large language models have achieved strong performance on programming benchmarks such as HumanEval [?], MBPP [?], and SWE-bench [?]. Self-repair approaches [? ?] improve correctness by iterating on test failures, while self-planning methods [? ?] decompose tasks before generation. *None of these works surface implicit assumptions or connect them to code regions for targeted revision.*

7.2 Requirements Engineering & Ambiguity

A rich body of RE research addresses natural-language ambiguity detection [? ? ?]. Classical approaches identify lexical or syntactic ambiguity but do not model the relationship between underspecified requirements and generated implementation decisions. *AssumptionMiner extends this line of work to the code-generation setting, where the artifact under analysis is executable code rather than a specification document.*

7.3 Traceability & Dependency Analysis

Traceability research [? ?] links requirements artifacts to code using information-retrieval or ML techniques. Static analysis tools [? ?] provide program-level dependency graphs. *AssumptionMiner combines semantic (LLM-based) dependency inference with lightweight AST analysis to produce the first assumption-to-code dependency graphs for LLM-generated artifacts.*

7.4 Interactive & Conversational Code Generation

Conversational repair tools [?] use test-driven iteration, while tools like Copilot rely on implicit context from the surrounding file. User studies [? ?] show that developers frequently struggle to identify *why* generated code is wrong. *AssumptionMiner directly addresses this gap by making implicit design decisions explicit and editable before they cause downstream failures.*

7.5 Specification Inference & Contract Mining

Automated specification mining [?] and contract inference techniques extract program invariants and behavioral contracts from executions or static analysis. LLM-augmented testing tools such as CodaMosa [?] and Fuzz4All [?] leverage language models to guide coverage-driven exploration and oracle generation. Unlike AssumptionMiner, all of these techniques operate *post-hoc* on existing code: they cannot attribute discovered properties to specific prompt ambiguities, nor do they surface assumptions *during* generation for developer review. AssumptionMiner is complementary—its structured assumption layer could *seed* downstream invariant checkers with candidate properties, but its primary goal is interactive disambiguation at generation time rather than post-hoc property discovery.

7.6 Architectural Decision Records

Architectural Decision Records (ADRs) [? ?] are lightweight documents that capture the rationale behind a design decision, its alternatives, and its consequences. The AssumptionRecord schema in AssumptionMiner is directly inspired by this tradition: both treat design decisions as first-class, addressable artifacts with explicit alternatives. The key distinction is automation: ADRs are authored manually by architects after the fact, whereas AssumptionMiner *generates* assumption records automatically during code synthesis, making the practice tractable for the routine, fine-grained decisions embedded in everyday LLM code generation.

7.7 Automated Program Repair

Automated Program Repair (APR) techniques [? ? ?] generate patches that correct program failures, typically guided by failing tests or formal specifications. The incremental regeneration component of AssumptionMiner (C5) shares structural similarities with patch generation: both isolate a minimal code region and apply a targeted transformation. However, APR is *fault-driven*—triggered by observable failures—while AssumptionMiner's regeneration is *assumption-driven*, triggered by developer preference changes on code that may already pass all tests (the working-but-wrong regime that APR cannot address). This makes C5 closer in spirit to requirement-driven change propagation than to defect localization and repair.

8 Threats to Validity

We assess threats following Wohlin et al. [?].

Internal validity. Annotation bias may affect AmbigBench ground truth: annotators were graduate students aware of the study goals. We mitigated this with calibration rounds and senior-author adjudication. LLM non-determinism (temperature > 0) means reported numbers reflect averaged runs; we report means over 3 runs per task (temperature=0 for extraction, 0.7 for regeneration).

LLM backbone bias. The assumption taxonomy and AmbigBench annotations were developed while using GPT-4o, and AssumptionMiner's extractor is also GPT-4o-based; evaluation on GPT-4o outputs thus carries a risk of circular self-consistency rather than true generalization. We partially mitigate this by (i) using held-out tasks not seen during taxonomy development, and (ii) reporting ablations with Claude 3.5 Sonnet and Gemini 1.5 Pro (see table 3), but

full independence would require a backbone-agnostic benchmark constructed without any LLM assistance.

Benchmark contamination. HumanEval [?] and MBPP [?] tasks are part of GPT-4o's training data; the model may therefore generate predictable, stereotyped solutions whose assumptions are easier to extract, potentially inflating extraction accuracy relative to novel tasks. The 50 novel tasks in AmbigBench are designed to be out-of-distribution for this reason, and we report per-source breakdowns to allow readers to assess this effect.

External validity. AmbigBench tasks are drawn from Python; results may not generalize to statically typed languages (e.g., Rust, Java) where some assumptions are enforced by the type system. Industrial-scale codebases with multi-file context may expose additional assumption categories not in our taxonomy.

Construct validity. Our five metrics (*AR*, *AP*, AF_1, *AS*, *CAS*) operationalize assumption quality as human-judged matches; they cannot capture all nuances of semantic equivalence. The SUS instrument [?] measures perceived usability, which correlates with but does not equal actual productivity gains.

Conclusion validity. The user study sample (≥20 participants) is sufficient for detecting medium-to-large effect sizes ($d \geq 0.5$, power ≥ 0.8) but may be under-powered for small effects. Although task order was counterbalanced across participants, within-subjects designs carry learning-effect risks: participants who encounter AssumptionMiner second may perform better on the Baseline condition due to improved familiarity with the task type itself, not the tool. We analyze order as a covariate in our statistical models to detect and control for this confound.

9 Conclusion

LLM code generation silently embeds a large number of implicit assumptions that developers have no systematic way to inspect or revise. These hidden decisions are a root cause of working-but-wrong code—code that passes tests yet violates developer intent.

We presented AssumptionMiner, a framework that mines implicit assumptions from LLM-generated code, surfaces them as structured *AssumptionRecord* objects linked to their governing code regions, and propagates developer-specified revisions through targeted incremental regeneration. On a 30-task pilot of AmbigBench, the best offline baseline achieves $F_1 = 0.12$ (pattern extraction), confirming that implicit assumptions require LLM-level semantic reasoning beyond surface syntax. AssumptionMiner's dependency mapper achieves 56.8% accuracy, outperforming the strongest baseline (KH, 53.2%) by 3.6 pp on 111 expert-annotated assumption–code links. Pipeline validation on all 30 tasks confirms 100% syntactically valid code after assumption revision, with AssumptionMiner's targeted regeneration producing the smallest edit distance (0.328) among non-regeneration methods—consistent with its incremental-update design hypothesis. Full LLM-based extraction results and a user study with ≥20 developers are in progress and will be reported in the final version.

Future work includes extending AssumptionMiner to multi-file repositories, incorporating project-level style guides as assumption priors, and studying how assumption mining interacts with test generation tools. We release AmbigBench and the AssumptionMiner implementation at `[anonymized for review]`.